

```
/**
 * mcp-universe SDK
 * The simplest way to connect AI to any MCP server.
 *
 * @example
 * import { MCPUniverse } from 'mcp-universe';
 * const universe = new MCPUniverse({ gateway: 'http://localhost:3000' });
 * const result = await universe.run('search the web for MCP news');
 */
```

```
import EventEmitter from 'eventemitter3';
```

```
// — Types —
```

```
export interface MCPUniverseOptions {
  /** Gateway URL, e.g. 'http://localhost:3000' */
  gateway: string;
  /** Optional API key for authenticated instances */
  apiKey?: string;
  /** Request timeout in ms (default: 30000) */
  timeout?: number;
}
```

```
export type Transport = 'sse' | 'stdio' | 'docker' | 'http';
```

```
export interface ConnectOptions {
  /** Unique name for this server connection */
  name: string;
  /** Transport protocol */
  transport: Transport;
  /** For SSE/HTTP: server URL */
  url?: string;
  /** For STDIO: command to run */
  command?: string;
  /** For STDIO: command arguments */
  args?: string[];
  /** For Docker: image name */
  image?: string;
  /** Environment variables for STDIO/Docker */
  env?: Record<string, string>;
}
```

```
export interface Tool {
  /** Tool name */
  name: string;
  /** Human-readable description */
  description: string;
  /** Which server provides this tool */
  server: string;
  /** Trust score 0-1 (higher = more reliable) */
  trust: number;
}
```

```

/** JSON Schema for input parameters */
inputSchema: Record<string, unknown>;
}

export interface Session {
  /** Session ID */
  id: string;
  /** Shareable URL */
  url: string;
  /** ISO timestamp */
  createdAt: string;
  /** Connected server names */
  servers: string[];
}

export interface RunResult {
  /** Final answer */
  answer: string;
  /** Tools that were used */
  toolsUsed: string[];
  /** Raw tool outputs */
  steps: ToolCall[];
}

export interface ToolCall {
  tool: string;
  args: Record<string, unknown>;
  result: unknown;
  durationMs: number;
}

export type UniverseEvents = {
  'tool:start': (data: { tool: string; args: Record<string, unknown> }) => void;
  'tool:done': (data: { tool: string; result: unknown; durationMs: number }) => void;
  'tool:error': (data: { tool: string; error: Error }) => void;
  'server:connected': (data: { name: string; transport: Transport }) => void;
  'server:disconnected': (data: { name: string }) => void;
};

// ——— SessionManager —————

export class SessionManager {
  constructor(
    private readonly baseUrl: string,
    private readonly headers: Record<string, string>,
  ) {}

  /** Save current session and get a shareable URL */
  async save(): Promise<Session> {
    const res = await this.fetch('/sessions', {
      method: 'POST',
      body: JSON.stringify({}),
    });
  }
}

```

```

    return res as Session;
}

/** Load a session by ID or full URL */
async load(idOrUrl: string): Promise<Session> {
    const id = idOrUrl.includes('/s/') ? idOrUrl.split('/s/')[1] : idOrUrl;
    const res = await this.fetch(`/sessions/${id}`);
    return res as Session;
}

private async fetch(path: string, init?: RequestInit): Promise<unknown> {
    const response = await globalThis.fetch(`${this.baseUrl}${path}`, {
        ...init,
        headers: { 'Content-Type': 'application/json', ...this.headers, ...(init?.headers ?? {}) }
    },
    });
    if (!response.ok) {
        throw new Error(`Session API error ${response.status}: ${await response.text()}`);
    }
    return response.json();
}
}

// ——— MCPUniverse —————

```

```

export class MCPUniverse extends EventEmitter<UniverseEvents> {
    private readonly baseUrl: string;
    private readonly headers: Record<string, string>;
    private readonly timeout: number;

    /** Session management helper */
    public readonly session: SessionManager;

    constructor(options: MCPUniverseOptions) {
        super();

        if (!options.gateway) {
            throw new Error('MCPUniverse: gateway URL is required');
        }

        this.baseUrl = options.gateway.replace(/\/$/, '');
        this.timeout = options.timeout ?? 30_000;
        this.headers = {
            'Content-Type': 'application/json',
            ...(options.apiKey ? { Authorization: `Bearer ${options.apiKey}` } : {}),
        };

        this.session = new SessionManager(
            `${this.baseUrl}/api`,
            this.headers,
        );
    }
}

```

```
/**
 * Connect a new MCP server to the universe.
 *
 * @example
 * await universe.connect({ name: 'files', transport: 'stdio',
 *   command: 'npx', args: ['-y', '@modelcontextprotocol/server-filesystem', '/tmp'] });
 */
async connect(options: ConnectOptions): Promise<void> {
  await this.post('/api/servers', options);
  this.emit('server:connected', { name: options.name, transport: options.transport });
}

/**
 * Disconnect a server by name.
 */
async disconnect(name: string): Promise<void> {
  await this.delete(`/api/servers/${encodeURIComponent(name)}`);
  this.emit('server:disconnected', { name });
}

/**
 * List all connected servers.
 */
async servers(): Promise<ConnectOptions[]> {
  return this.get('/api/servers') as Promise<ConnectOptions[]>;
}
```

// — Tool management

```
/**
 * List all available tools across all connected servers,
 * sorted by trust score (highest first).
 *
 * @example
 * const tools = await universe.tools();
 * // [{ name: 'read_file', server: 'files', trust: 0.98 }, ...]
 */
async tools(): Promise<Tool[]> {
  const tools = await this.get('/api/tools') as Tool[];
  return tools.sort((a, b) => b.trust - a.trust);
}

/**
 * Call a specific tool by name.
 *
 * @example
 * const result = await universe.call('read_file', { path: '/tmp/hello.txt' });
 */
async call(toolName: string, args: Record<string, unknown> = {}): Promise<unknown> {
  const start = Date.now();
  this.emit('tool:start', { tool: toolName, args });
}
```

```

    try {
      const result = await this.post('/api/tools/call', { tool: toolName, args });
      const durationMs = Date.now() - start;
      this.emit('tool:done', { tool: toolName, result, durationMs });
      return result;
    } catch (error) {
      this.emit('tool:error', { tool: toolName, error: error as Error });
      throw error;
    }
  }
}

```

// — AI Orchestration —————

```

/**
 * Run a natural language query. The universe automatically selects
 * and calls the right tools to answer your question.
 *
 * @example
 * const result = await universe.run('search the web for MCP news');
 * console.log(result.answer);
 */
async run(query: string): Promise<RunResult> {
  return this.post('/api/run', { query }) as Promise<RunResult>;
}

```

// — Health —————

```

/**
 * Check gateway health. Returns true if the gateway is reachable.
 */
async health(): Promise<boolean> {
  try {
    const res = await globalThis.fetch(`${this.baseUrl}/health`, {
      signal: AbortSignal.timeout(5000),
    });
    return res.ok;
  } catch {
    return false;
  }
}

```

// — Static helpers —————

```

/**
 * Create a new MCPUniverse instance from a shared session URL.
 *
 * @example
 * const universe = await MCPUniverse.fromSession('https://my-instance/s/abc123', {
 *   gateway: 'http://localhost:3000'
 * });
 */
static async fromSession(

```

```

    sessionUrl: string,
    options: MCPUniverseOptions,
  ): Promise<MCPUniverse> {
    const universe = new MCPUniverse(options);
    await universe.session.load(sessionUrl);
    return universe;
  }

// — Private helpers —————

private async get(path: string): Promise<unknown> {
  const response = await globalThis.fetch(`${this.baseUrl}${path}`, {
    headers: this.headers,
    signal: AbortSignal.timeout(this.timeout),
  });
  if (!response.ok) {
    throw new Error(`MCP Universe API error ${response.status}: ${await response.text()}`);
  }
  return response.json();
}

private async post(path: string, body: unknown): Promise<unknown> {
  const response = await globalThis.fetch(`${this.baseUrl}${path}`, {
    method: 'POST',
    headers: this.headers,
    body: JSON.stringify(body),
    signal: AbortSignal.timeout(this.timeout),
  });
  if (!response.ok) {
    throw new Error(`MCP Universe API error ${response.status}: ${await response.text()}`);
  }
  return response.json();
}

private async delete(path: string): Promise<void> {
  const response = await globalThis.fetch(`${this.baseUrl}${path}`, {
    method: 'DELETE',
    headers: this.headers,
    signal: AbortSignal.timeout(this.timeout),
  });
  if (!response.ok) {
    throw new Error(`MCP Universe API error ${response.status}: ${await response.text()}`);
  }
}

// — Convenience exports —————

export default MCPUniverse;

```